SP-108 Green ASM

SDS

CS 4850 - Section 01 – Spring 2026

2/8/2026 Professor Perry

Alex Tidwell Team Leader Documentation + Developer (as needed)	Katherine Smith Documentation + Developer	Sumaya Tamenne Documentation	Phillip Eiser Developer	Brianna Lewis Developer

Table of Contents

1. INTRODUCTION AND OVERVIEW 3

2. DESIGN CONSIDERATIONS 3

2.1. ASSUMPTIONS AND DEPENDENCIES 3
2.2. GENERAL CONSTRAINTS 3
2.3. DEVELOPMENT METHODS 4

***3.* ARCHITECTURAL STRATEGIES 4**

3.1. TOOL ORCHESTRATION AND INTEGRATION STRATEGY 4
3.2. USE OF EXISTING SECURITY TOOLS 5
3.3. EXTERNAL DEPLOYMENT AND NETWORK SEPARATION STRATEGY 5
3.4. DATA STORAGE AND HISTORICAL ANALYSIS STRATEGY 5
3.5. VISUALIZATION AND INTERFACE STRATEGY 5
3.6. PLATFORM AND INFRASTRUCTURE SELECTION 5
3.7. FUTURE SYSTEM EXTENSION CONSIDERATIONS 6
3.8. REPORTING AND NOTIFICATION STRATEGY 6

4. SYSTEM ARCHITECTURE 6

4.1 OVERVIEW 6
4.2 ASM ENGINE SUBSYSTEM 6
4.3 TARGET ENVIRONMENT SUBSYSTEM 7
4.4 DATA STORAGE AND REPORTING COMPONENT 7

5. DETAILED SYSTEM DESIGN 7

5.1. CLASSIFICATION 7
5.2. DEFINITION 7
5.3. CONSTRAINTS 7
5.4. RESOURCES 8
5.5. INTERFACE/EXPORTS 8

6. GLOSSARY 8

7. BIBLIOGRAPHY 9

1. Introduction and Overview

This document describes the design plan for our automated Attack Surface Management (ASM) system. The primary goal of the system is to continuously discover, analyze, and monitor exposed vulnerabilities within a target environment. Unlike traditional ASM systems that operate on a known list of assets, this system will approach the environment from an attacker's viewpoint, locating assets that may be unknown or unintentionally exposed. The system will use many open-source security tools to perform asset discovery, network scanning, and vulnerability detection. Scan results will be stored and used for comparison continuously to detect any changes in the attack surface. This document focuses on the system's primary design plans and decisions, including assumptions, constraints, and the development approach.

2. Design Considerations

Several factors impact the design and implementation of the ASM system, including technical, environmental, and operational factors. The considerations below ensure the system remains realistic in its limitations while also remaining highly functional.

2.1. Assumptions and Dependencies

The design of this project assumes the availability of at least two Linux virtual machines to run the system. One serves as the target environment, and the other as the scanning environment, simulating an attacker. The system also depends on Docker being installed and configured correctly in the scanning environment to support container use for major components. The system's functionality depends on several open-source tools. OWASP Amass will be used for asset discovery, Nmap for network scanning, Watchtower for container monitoring, Nuclei for vulnerability detection, and reNgine for workflow management.

The system assumes a strict Linux environment, as the open-source tools and containerized architecture require Linux for full functionality. It is assumed that the scanning system will be able to communicate with the target environment and the internet as needed. This is required for tasks such as asset discovery and vulnerability scanning.

This project is created for users with knowledge of Linux systems, Docker, and core cybersecurity concepts. Users are expected to understand how to access environments via SSH, manage containers, and use web-based dashboards to interpret scanning results. Lastly, the design assumes that the ASM's functionality may change in the future. Tools used for scanning, vulnerability templates, and management platforms are susceptible to change, which would require updates and configuration adjustments. Additional features, including expanded reporting capabilities and deeper vulnerability enrichment, may be introduced over time.

2.2. General Constraints

The design of this system is constrained by several technical, environmental, and operational factors that affect its behavior and performance. As stated, the software is intended for a

Linux-based environment because it uses tools and Docker containers. System resources, including CPU, memory, and storage, are additional constraints, as execution platforms such as reNgine require up to 8 GB of RAM and adequate disk space to perform reliably. These constraints affect scan frequency, execution speed, and long-term data maintenance.

End-user constraints impose additional constraints on the system because it is designed for controlled environments and authorized users. Access to the scanning environment must be protected with SSH key authentication, limiting system access to authorized users only. Another constraint involves network conditions. As stated, communication between the two environments is needed to enable vulnerability scans. Scan completeness can also be affected by factors such as firewalls, segmentation, or other restricted-access tools.

Integration constraints arise because several tools must work together for optimal performance. While execution platforms simplify integration complexity, scan results still need to be normalized to support the report's format. Lastly, there are constraints regarding the storage of historical reports. Since data must be continuously stored, avoiding unnecessary replication of large external datasets is essential for efficiency.

2.3. Development Methods

This system will be developed using a design approach that emphasizes focusing on ease of implementation and the use of open-source security tools. The design process involved researching and selecting tools best suited to the project's functionality. When selecting tools, the primary focus was on tasks such as asset discovery, vulnerability detection, and tool integration. Docker on a virtual machine was chosen as the deployment method, as it provides a consistent environment and allows all team members to access containers individually.

Tools were researched and lightly tested during the decision process to ensure proper integration. When creating the design, reNgine was chosen as the development approach to simplify development, reduce system complexity, and provide a unified interface for managing scans and reports. Using this tool supports a maintainable implementation while still allowing for optimal performance and results. Lastly, an incremental and iterative development approach will be used for this system. This way, individual parts will be tested and configured independently before being integrated with other tools.

3. Architectural Strategies

3.1. Tool Orchestration and Integration Strategy

The main tools that the ASM system will use are OWASP Amass, Nmap, Nuclei, and reNgine, which will all work in pipeline coordination. OWASP Amass will work to enumerate the domain and subdomains of the target enclave. Then Nmap can work for network mapping, which would include discovering open ports and active services. Nuclei would help discover any vulnerabilities in the system. Lastly, the tool reNgine is important for piecing all these tools together and giving the team a dashboard to help visualize, organize, and keep active reports.

The language chosen to help with extracting data, comparison of scans, and communication with the team was Python, which will be used to write scripts that will automate these tasks. Python comes with a robust library and provides plenty of resources regarding script work at this level.

3.2. Use of Existing Security Tools

The ASM system is designed around the use of open-source tools to perform active scans of a company's attack surface and identify any vulnerabilities. The use of these open-source tools was chosen to help focus on analysis, correlations, and orchestration, which is the main purpose of an ASM system. By using these tools, the ASM will be able to perform asset discovery, vulnerability detection, and network scanning, simulating real-world scenarios.

An alternative to open-source tools was building custom discovery vulnerability scanning tools; however, this would shift the project's focus to low-level computation rather than system design. For this project, as well, this would cause more complexity and risk, which would take away from the main goal of analysis. These open-source tools offer lots of flexibility with how the project team sets up our system, and also allow the team to add to them if future plans call for this.

3.3. External Deployment and Network Separation Strategy

For the system architecture, the ASM system is designed to run on a separate system and network to simulate a real attacker. This allows the ASM system to detect possible vulnerabilities that an attacker would see. Also, in real-world scenarios, the attack surface for companies can extend across networks and cloud services, meaning detection does not happen solely on the same system. This separation will enable the detection of services provided and test the tools being used to their full capacity.

3.4. Data Storage and Historical Analysis Strategy

For the data storage architecture, there will be a dedicated CSV file for storing asset inventory, a file for storing raw data of what the scans detected, a file for the drift reports detecting the differences, and a file for storing CVE references associated with detected vulnerabilities. This method of data storage was chosen for its simplicity and ease of use for the scope of this project. By using a CSV for asset inventories and other storage, maintaining an accurate history for comparisons is easier and more flexible with the scripts the system will be using. If we decide we need more structure and easier querying we will go with a relational database

3.5. Visualization and Interface Strategy

The system will be orchestrated through reNgine, which will act as an interface for the team to interact with when running scans and gathering the raw data from the scans. This tool will help the team with the visualization of known assets and vulnerabilities while also helping to avoid the use of unnecessary custom API integration.

3.6. Platform and Infrastructure Selection

There will be two separate virtual machines, one for orchestration and scanning, the other to hold the Docker containers that run the various services. Docker was chosen as the optimal choice for its ease of use with tools like CoreDNS for setting up a DNS server, as well as the ease to deploy services in different containers. This allows expansion as the ASM machine is further improved upon and demands more testing.

Linux as the operating system is a strong option for this project as it offers the most accessibility for the tools being used and is a common choice for running up services such as those for the target. As well as running the tools needed for the ASM target. Kali Linux is

considered due to many of these tools coming preinstalled, though other distributions are still up for debate depending on deployment needs. These features, however, further the reasoning for using the Linux operating system.

3.7. Future System Extension Considerations

With the tools being used and how the system is set up, there is a possibility to improve beyond what is being worked on right now. Adding further prebuilt tools could be a possibility as well, improving upon the already used open-source tools. Also, building a custom UI specific to this ASM machine would be something that would add a lot of ease when trying to operate the system. Adding an integrated database within this custom UI dashboard would also help, as the more assets and services that are added to the scope, the more the CSV files can confuse. A custom database would give teams the ability to look through scans faster and find correlations to history more easily.

3.8. Reporting and Notification Strategy

An important part of the architecture is receiving info from these scans that alerts when they happen and what info is being received. Simple Python scripts can offer a way to send outputs to Discord webhooks. Tools such as Watchtower may be used, which offer the ability to monitor any changes in the Docker containers and detect unexpected changes or drift. These two offer a solid way to receive notifications and updates on the system.

4. System Architecture

This machine is built to be an Attack Surface Management system, which will contain an external ASM engine, a data storage layer, a User Interface layer, and a reporting layer. The target is also included, which will contain the assets needed to properly test the ASM system. This system will have the ability to detect vulnerabilities, map out subdomains, log assets, and monitor changes through network scanning.

4.1 Overview

The architecture contains two separate virtual machines running Linux operating systems. The first virtual machine will contain the ASM system, which is responsible for deploying tools, storing results from these tools, and generating reports that base scans on severity and what was specifically seen. The target environment will be run on a separate machine and will be responsible for simulating a company's attack surface. It will contain separate containers with different services and is assigned by mapping out these services to different subdomains for the ASM system to detect.

4.2 ASM Engine Subsystem

The ASM engine has four separate functions, which will run in a pipeline. These include discovery, network scanning, vulnerability detection, and monitoring. There is an orchestration layer that helps to combine all these functions and provide scan reports. Then there will be custom scripts to help take this data and create reports, perform comparisons, and handle reporting tasks. This subsystem is the main system that provides the tools and analysis for the entire project.

4.3 Target Environment Subsystem

The target environment uses Docker to have multiple containers simulating companies' subdomains as stated before. This subsystem is in charge of running the various services and network functions. It will have multiple vulnerabilities and misconfigurations that could be detected.

4.4 Data Storage and Reporting Component

All the scans will be stored so historical context can be tracked. This data will be able to show the various changes in assets, services, and vulnerabilities. From this drift reports and asset inventories will be stored in a separate area to be reviewed.

5. Detailed System Design

5.1. Classification

The ASM is security software that will be able to scan a network for vulnerabilities and provide in-depth reports about those vulnerabilities, including things like matching CVEs and a CVSS score, and then store these reports in a database. We will have two VMs, one for our enclave component and one for our ASM component. We will have a third VM or more for our attacker component when it comes time to test our system. The actual ASM itself will have three overall components, a scanning component, an orchestration component, and a report and storage component.

5.2. Definition

The ASM is security software that will be able to scan a network for vulnerabilities and provide in-depth reports about those vulnerabilities, including things like matching CVEs and a CVSS score, and then store these reports in a database. The enclave component is a VM that we will run multiple Docker containers out of that interact with each other and form a sort of DNS network. Our ASM component will run our system scans on the network created on the first VM for vulnerabilities, generate the reports, show results in a dashboard, as well as store them in the database. The attacking component will present a possible attack to this network that will try to do things to get around our ASM's alert system, so that we can test and improve things where we need to. The scanning component will involve using OWASP Amass for subdomain enumeration and Nmap for network mapping/scanning. The orchestration component will be running ReNgine so that we can have an interface to coordinate these tools in a way that lets them function together and do what we need them to. Finally, the report and storage component will involve a dashboard that allows us to run scripts, either when we want or set it up to be automated, that will run our scans and generate a report that displays relevant information, like matching CVEs and CVSS scores, to the dashboard and sends the information to our database for storage. Our database will either be in the form of a relational database or a simpler CSV file.

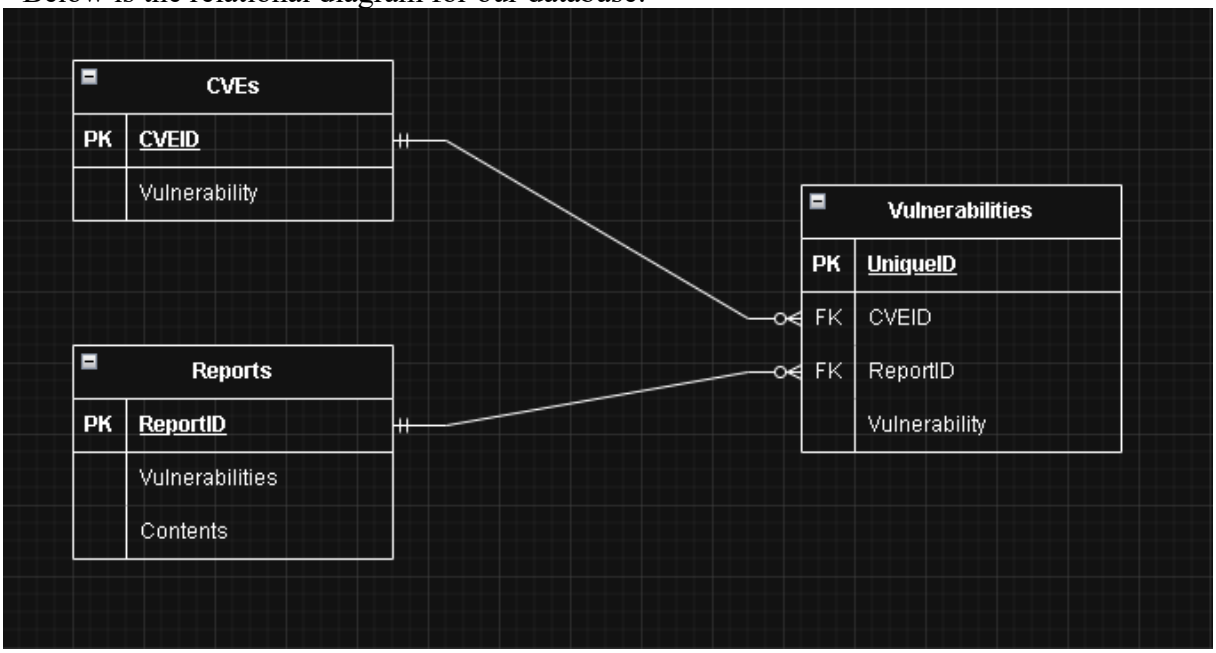
5.3. Constraints

The only real major constraints about our components are that we should not have multiple users editing the same thing in the enclave or ASM at the same time, and that if the cloud service we are using for our VMs goes down we are not able to work in those VMs at that time. A possible constraint for the database component could be if we decide to use a CSV

file instead of a full database to store out information, there is a much higher chance of data accidentally being overwritten by multiple people trying to run a scan at the same time.

5.4. Resources

Our project will need a handful of resources. We will need two VMs, one for running our enclave target and the other for our ASM and we will require these to be on the cloud so that any member of our group can access them at any time, as well as there be a possibility of regular scanning so we can test that our scripts are running properly and give it the opportunity to detect vulnerabilities we may not have created. We will eventually also need local VMs for our red phase so that we can test the system. We will need to have a database to store our drift reports, vulnerabilities found, and CVEs they relate to for cross-reference. Below is the relational diagram for our database.



5.5. Interface/Exports

The ASM component will run scripts to start the scanning of the network and provide the information that is gathered from the scan, including any possible vulnerabilities it finds and create a drift report that the user can see by using the dashboard. This component will also have a script that adds the report and vulnerabilities found to the database for storage and future querying. The dashboard will also be able display queries about previous reports or information about vulnerabilities.

6. Glossary

The following definitions and acronyms are provided to explain terminology used throughout the document:

- **Attack Surface Management (ASM):** the process of continuously discovering and monitoring all exposed digital assets in an organization's attack surface.
- **Attack Surface:** all exposed digital assets, such as IP addresses and servers, domains and subdomains, APIs, cloud resources, and open ports.

- **Asset:** property owned by an individual or organization that has value.
- Common Vulnerabilities and Exposures (CVE): a list of publicly disclosed cybersecurity vulnerabilities.
- **National Vulnerability Database (NVD):** U.S government-maintained database that provides information on CVEs
- **Active Probing:** the process of sending network requests to services to test their behavior and acknowledgements.
- **Port Scanning:** the process of discovering open ports to verify accessible services through the given port.
- **Red Phase:** A controlled system that simulates an attack environment by introducing vulnerabilities to test the function of the ASM.
- **Container:** an isolated runtime environment that wraps an application and any dependencies.
- **Virtual Machine (VM):** a virtual computer environment that runs any specified operating system that acts as a physical machine.
- **Common Vulnerabilities and Exposures (CVE):** a list of public security flaws in computing.
- **Domain Name System (DNS):** how a computer looks up and find corresponding IP addresses with their domain names.
- **CSV:** a plaintext file that stores tabular data in a format separated by commas

7. Bibliography

Am, M. I. (2025, October 21). *Amass 5.0.0 Usage for Recon*. Medium. https://medium.com/@marduk.i.am/amass-5-0-0-usage-for-recon-8041bc727480

amass | Kali Linux Tools. (n.d.). Kali Linux. https://www.kali.org/tools/amass/

Anamar Taknev. (2024, August 5). *Beginners guide to amass – Hackercool Magazine*. Hackercool Magazine. https://www.hackercoolmagazine.com/beginners-guide-to-amass/?srsltid=AfmBOooWfPNIBIO9HpFRqYTWkQaEiNpVSZpMLk6j5pIWy6FBBZcp_182

Nmap. (2019). *Chapter 15. Nmap reference guide | nmap network scanning*. Nmap.org. https://nmap.org/book/man.html

nmap | Kali Linux Tools. (n.d.). Kali Linux. https://www.kali.org/tools/nmap/

OWASP Amass Contributors. (2017). *OWASP Amass Project*. Github.io. https://owasp-amass.github.io/docs/

Running Nuclei - ProjectDiscovery Documentation. (2020). Projectdiscovery.io; ProjectDiscovery Documentation. https://docs.projectdiscovery.io/opensource/nuclei/running

Watchtower. (2025). Containrrr.dev. https://containrrr.dev/watchtower/